

1 System Architecture

1.1 Overview

The architecture of the Bug Severity Classification System is divided into two major layers:

- Software Level (Application Layer)
- MLOps Level (Pipeline and Monitoring Layer)

This separation ensures modularity, scalability, and maintainability of the system.

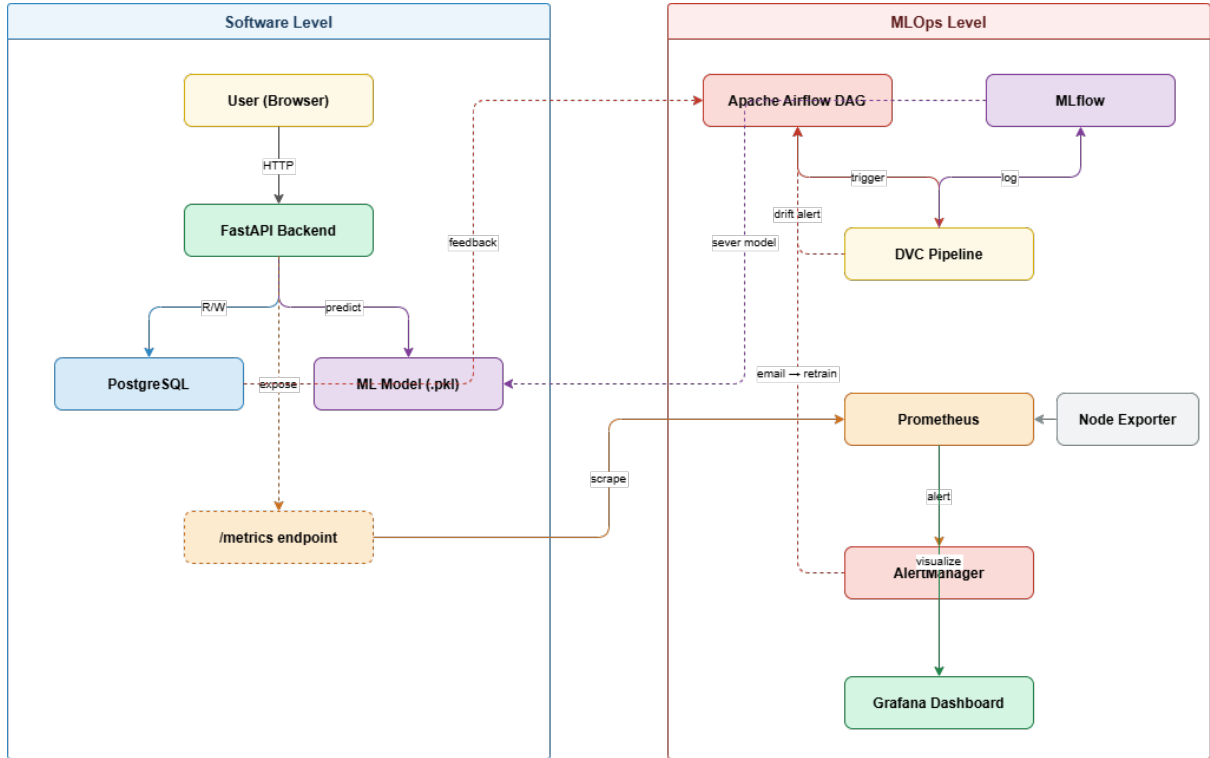


Figure 1: System Architecture of Bug Severity Classification Application

1.2 Software Level

The software level represents the user-facing application and core backend services.

1.2.1 User Interface

The user interacts with the system through a browser. Requests are sent to the backend via HTTP.

1.2.2 FastAPI Backend

The backend is implemented using FastAPI and acts as the central controller of the system. It performs the following tasks:

- Receives user requests
- Processes input data
- Calls the machine learning model for prediction
- Communicates with the database
- Exposes metrics for monitoring

1.2.3 Database (PostgreSQL)

PostgreSQL is used for storing:

- Bug reports
- Prediction results
- Feedback data

1.2.4 Machine Learning Model

A trained machine learning model (stored as a `.pkl` file) is used to classify bug severity.

1.2.5 Metrics Endpoint

The backend exposes a `/metrics` endpoint for monitoring.

1.3 MLOps Level

1.3.1 Apache Airflow

Airflow orchestrates workflows and triggers pipelines.

1.3.2 DVC Pipeline

DVC manages data versioning and reproducibility.

1.3.3 MLflow

MLflow tracks experiments and manages models.

1.3.4 Monitoring Stack

Includes Prometheus, Node Exporter, Grafana, and AlertManager.

1.4 System Workflow

1. User sends request
2. FastAPI processes and predicts
3. Data stored in PostgreSQL
4. Metrics exposed
5. Prometheus scrapes metrics
6. Grafana visualizes
7. Alerts trigger retraining